Entity-Oriented CRDT Documents: Architecture, Identity, and Structural Liveness

Merkle, B.[1] Rodionov, V.[2]

Draft — preprint, April 2026

Abstract

CRDT documents are conventionally modeled as nested trees: containers embed containers, and an entity's identity is its position in the tree. This model composes poorly with three pressures of entity-oriented applications — multi-parent references, stable identity through restructuring, and reachability-based liveness. We propose a document model in which entities are flat shells in type-indexed registries, and parent→child relationships are decodable UUID references rather than direct embedding. The reference graph becomes a first-class structural object: enumerable from schema, traversable in $O(\log m)$ per hop via coordinate-derived identity, and usable as a liveness signal. We describe a concrete 15-character identifier encoding with lifecycle-discriminated prefixes, Feistel visual dispersion, and a URL/JS/CSS-safe alphabet. We show that this model admits a GC alternative that replaces tombstone-based deletion with reachability-driven cold-storage offload, sidestepping Dolan's impossibility result [1] for algebraic undo of general CRDTs. Deployed in a production framework (Plexus/Yjs); Appendix A documents the Yjs-specific undo workarounds required to preserve identity stability under local undo/redo.

1 Introduction

CRDT research has optimized the convergence contract: given the same set of operations, peers reach identical state. For text, sequences, and small structured documents this is sufficient. Entity-oriented applications — design tools, project management, visual programming environments — place pressure on the nested-tree document model in ways that convergence alone does not address.

Multi-parent references. A style token is referenced from many components. A design page appears in multiple navigation menus. The same entity is reachable through several paths in the logical hierarchy. In a nested-tree model this forces either duplication (each parent gets a copy, losing shared identity) or an external symbol table (paying all the costs of an auxiliary registry).

Identity stability through restructuring is the second pressure: moving a subtree should preserve the identity of its entities — their references from elsewhere in the document, their addresses in dependency bundles, their presence in snapshots taken before the move. Position-derived identity fails this, because a move is a new insertion under the new parent, the creating operation's coordinates change, and identity drifts.

Reachability-based liveness. Deleting an unreferenced entity is safe only if we can *enumerate* references to check. In a nested-tree model, references are opaque strings embedded in arbitrary fields — application-managed registries of reference-bearing paths are required, and they rot. Without cheap enumeration, the system must either retain every entity forever (tombstones growing unboundedly) or accept orphan bugs.

We present a document model that makes all three tractable. The contribution is architectural: completeness checking, partial replication, and structural GC follow as direct consequences of a structural decision about how entities are stored, rather than as independently engineered features. The paper then describes the concrete identifier encoding the model depends on in practice (§5) and documents the Yjs-specific runtime discipline required to keep identity stable across undo/redo (Appendix A).

2 The Entity-Oriented Document Model

2.1 Flat Shells in Type-Indexed Registries

The document's top level is a map of type names to entity registries:

```
doc
|-- types
```

[1]Independent researcher, Here.build; x.com/merkle_bonsai; Immunefi top–40 whitehat (ethical hacker); email: merklebonsai@pm.me

[2]Independent researcher, Here.build; email: v@here.build

```
|   |-- Component       <- type sub-map (one per entity
    class)
|   |   |-- uuid_a -> Shell { name: "Header",
|   |   |               children: [uuid_c, uuid_d] }
|   |   `-- uuid_b -> Shell { name: "Footer",
|   |                   children: [uuid_e] }
|   |-- Frame
|   |   `-- uuid_f -> Shell { title: "Home",
|   |                   root: uuid_a }
|   `-- StyleToken
|       `-- uuid_g -> Shell { name: "accent",
|                       value: "#3366ff" }
`-- meta                <- well-known pointers (root
    entity, ...)
```

Each type sub-map contains `uuid` → `shell` entries; shells are themselves CRDT containers (Y.Map or Y.XmlElement in our Yjs implementation). The type sub-maps are created via deterministic genesis[2] — they exist on every peer at identical addresses, without coordination, before any user operation.

Shells store field values. **Reference fields hold UUID strings, never embedded shells.** A Component's `children` field is an array of UUIDs that resolve to other entries (possibly in a different type sub-map). A Frame's `root` field is a single UUID. A StyleToken's `override` field is a UUID pointing to a parent token.

2.2 Why Flat?

The type-indexed flat layout is not cosmetic — it's what makes the three pressures of §1 tractable.

Multi-parent is structural. The same UUID can appear in multiple parent shells' reference fields with no duplication. A StyleToken referenced by fifty Components has exactly one shell in `types.StyleToken[uuid]`; the references are fifty UUID strings in fifty parent fields. Sync, snapshot, and resolution cost scales with unique entities, not reference count.

Identity is restructure-invariant. A shell's coordinates in the CRDT operation log are determined by when it was *created*, not where it is *parented*. Moving an entity from parent A to parent B is two field writes: remove the UUID from A's reference list, add it to B's. The shell's UUID, address, and cross-references stay identical.

The reference graph is a first-class object. Schema declares which fields are references (§3.1); decodable identity (§5) means every reference resolves to a specific shell without a lookup table. Graph traversal (§3.2) is $O((E + R) \cdot \log m)$ where E is entity count, R is reference count, m is per-client operation count. No auxiliary data structures.

2.3 Decodable References

References are not arbitrary strings. Each UUID is a self-resolving identifier that encodes a CRDT operation's `{clientId, clock}` pair in a compact fixed-length string. Given the operation log, any peer can decode the UUID and binary-search to the corresponding shell in $O(\log m)$. No lookup table, no registry, no index.

The architecture abstracts over the specific encoding — any fixed-length, decodable, coordinate-addressed scheme suffices. Automerge objectIds would qualify with adaptations; Yjs's `RelativePosition` is the position-level analogue. §5 gives the concrete encoding we use in Plexus (15 characters including a lifecycle prefix); §3 and §4 are encoding-agnostic.

For cross-document references, the identifier is extended to a tuple `[uuid, depId]` where `depId` routes to the dependency document.

3 The Reference Graph

3.1 Enumeration via Schema

The schema declares which fields contain entity references. In Plexus, this is determined by decorator annotations:

The set of reference-bearing fields is known at compile time. No runtime introspection needed.

Schema coverage caveat. The enumeration is sound (no false negatives) only when the schema covers all reference-bearing fields. User-extensible metadata or untyped plugin data may contain references invisible to the schema. At less than full coverage, enumeration provides a lower bound on the true reference set.

3.2 Traversal

Given a document and a root entity, compute the set of reachable entities and missing references:

```
traverse(doc, schema, root):
  visited <- {}
  missing <- {}
  queue <- [root.uuid]

  while queue is not empty:
    id <- queue.dequeue()
    if id in visited: continue
    visited.add(id)

    entity <- resolve(doc, id)
    if entity is null:
      missing.add(id)
      continue
```

Decorator	Reference type	Traversal
`@syncing accessor ref: Entity`	Scalar reference	Single tuple
`@syncing.list accessor refs: Entity[]`	Reference list	Array of tuples
`@syncing.child accessor child: Entity`	Ownership reference	Single tuple
`@syncing.child.list` / `.map` / `.set`	Ownership collections	Collection of tuples
Map keys with `EntityRef`[3]	Key-embedded references	Deserialize key, extract refs

```
  for each ref in enumerateReferences(entity, schema):
    if ref is local [refId]:
      queue.enqueue(refId)
    if ref is cross-document [refId, depId]:
      if depId not loaded:
        missing.add((refId, depId))
      else:
        queue.enqueue(refId)  // resolve in dep's doc

return { visited, missing }
```

`enumerateReferences` walks the entity's schema-declared fields and collects reference tuples. For composite map keys[3], it deserializes key strings and extracts references from key constructors.

Complexity. $O((E + R) \cdot \log m)$ where E is entity count, R is total reference count, m is per-client operation count for resolution. Cross-document references against in-memory dependency maps resolve in $O(1)$.

Cycle handling. The `visited` set prevents re-traversal of entities in cyclic reference graphs. For cross-document cycles (A $\rightarrow$ B $\rightarrow$ A), the traversal additionally tracks loaded documents.

4 Structural Liveness

The traversal of §3.2 produces two sets: `visited` (reachable entities) and `missing` (dangling references). The `missing` set is what drives partial-replication loading (§6). The `visited` set is what drives liveness.

Reachability as the liveness signal. An entity in the `visited` set is reachable from the root via some chain of ownership, reference, or key edges — the application can still access it through schema-declared paths. An entity in the CRDT operation log but absent from `visited` is structurally unreachable: no edge in the schema-enumerable reference graph leads to it. In a nested-tree model this signal is unavailable cheaply; here it is produced by the same traversal that supports completeness checking.

Deleting unreachable entities would be the obvious next step, and it is unsafe for CRDTs. Dolan[1] proves that algebraic undo of general CRDTs is impossible beyond counters: there is no compensating operation that reverses an arbitrary delete without losing information. If we delete an entity and a concurrent operation on another peer adds a reference to it, convergence is broken; if we delete and later want to restore via undo, the creating operation is gone. Tombstone-based CRDTs sidestep this by keeping deleted state forever, paying monotone storage growth for correctness. The options have been binary: either the storage grows with lifetime entity count (tombstones) or the system accepts convergence bugs around deletion (pragmatic GC with no theory).

Cold-storage offload as the disposition. The entity-oriented model admits a third option. Because shells are flat (independently addressable) and references are decodable (portable across storage tiers), an unreachable entity is serializable without its context — its UUID encodes its own coordinates, its content is self-contained in a single type-sub-map entry. We can *offload* rather than delete: remove the shell from the hot working snapshot, archive it in a cold store indexed by UUID, retain the operation-log entry for the creating operation (so coordinates remain decodable). The working set stays bounded by the reachable graph; lifetime storage grows slowly into a cold tier that can live on cheap storage, be garbage-collected on a much longer horizon, or be shared as an append-only history archive. When a UUID re-surfaces — a dependency bump, an undo that re-attaches a detached subtree, a stale reference in a snapshot — the cold store is consulted and the shell is promoted back into hot state. Dolan's constraint does not apply because nothing was undone: offload-promote is a lossless tiering cycle, not a state transition requiring a compensating operation. Tombstones become unnecessary because identity never expires; the operation log never shrinks, but the hot working snapshot does.

Distributed agreement is softer under offload than under deletion. Local reachability is necessary but not sufficient for any disposition: another peer may hold a reference we have not yet synced. Deletion-based GC must wait for causal stability[4] before acting — a missed reference causes irrecoverable data loss. Offload is more forgiving: an entity mistakenly offloaded because its reference was unsynced simply pays a cache miss

when the reference arrives, and the cold-store fetch promotes it back. Liveness becomes a performance-tier decision rather than a convergence-critical one. The causal-stability protocol remains useful for *compaction* of the cold tier on long horizons, but no longer gates correctness of the working set.

Completeness checking (what's missing), partial replication (what to load), and structural liveness (what to tier) are therefore three consequences of the same traversal — one graph walk serving three systems concerns, rather than three independently engineered mechanisms.

5 Identity Encoding

§2–§4 treat identifiers as abstract decodable strings. We now describe the concrete encoding deployed in Plexus, which is the encoding the appendix's undo discipline and the §3.2 traversal both assume in practice. It is not the only possible encoding for this architecture — any fixed-length coordinate-derived scheme would work — but it is the one we know satisfies the practical requirements simultaneously.

5.1 Requirements

To serve as an entity reference in this architecture, an identifier encoding must satisfy several practical constraints at once:

- **Self-resolving.** The string decodes to a specific `{replicaId, clock}` pair without a lookup table; the CRDT's native operation search resolves to the shell.
- **Fixed length.** References appear in field values, map keys, serialized snapshots, URLs, and class names. Variable-length identifiers make storage sizing, wire formats, and string handling awkward.
- **Lifecycle-discriminated.** Entity lifecycle stages (user-created, ephemeral preview, genesis, cloned) need to be distinguishable without fully decoding the identifier. Routing filters (exclude genesis from undo, strip ephemeral from persistence) are a per-operation hot path.
- **Safely embeddable.** References land inside JSON, JavaScript identifiers (`obj.pRxK4mN7pQ2wJbL`), CSS class names (`.pRxK4mN7pQ2wJbL { ... }`), URL path segments, and HTML attribute values. No escaping for common serialization contexts.
- **Visually dispersive.** Sequentially created entities produce sequentially adjacent `{clientId, clock}` pairs. If the encoding is order-preserving, consecutive references differ only in trailing characters — making diffs noisy and visual identification hard.
- **Identity-stable through local undo.** If undo retracts the creating operation, a naïve decoder would produce dangling references (if undo succeeds) or assign fresh identity on redo (if undo is suppressed). Appendix A describes the runtime discipline that resolves this.

Prior coordinate-derived encodings satisfy a subset. Automerge's `objectId` is variable-length and single-namespace; Yjs's `RelativePosition` / Automerge `Cursor` / Loro `ContainerID` operate at the position or container level rather than surfacing application-level entity identifiers. We present an encoding that meets all the requirements together.

5.2 The Encoding

An entity identifier is a 15-character ASCII string:

```
+- prefix (1 char): lifecycle stage
|  +- body (14 chars): base-63 encoded
|  |  {clientId, clock}
|  |
p  RxK4mN7pQ2wJbL
```

Prefix alphabet:

Prefix	Lifecycle	ClientId Range	Derivation
`p`	User-created	Regular $[0, 2^{51})$	range
`l`	Liminal preview	Liminal $[2^{51}, 2^{52})$	range
`b`	Bound (clone)	Regular $[0, 2^{51})$	1-bit flag
`d`	Deterministic	Genesis $[3 \cdot 2^{51}, 2^{53})$	range

Three of the four prefixes (`p`, `l`, `d`) are one-to-one with clientId ranges defined by the companion namespace partitioning scheme[5]. The fourth (`b`) is different, and we want to be precise about how. A bound entity has the same underlying clientId range as a persistent one (both in $[0, 2^{51})$); the `b`/`p` distinction is a **1-bit lifecycle flag** the encoder is told at encode time via a `binding` argument. Formally, the encoding is not a pure function of `{clientId, clock}` — it is a function of `{clientId, clock, binding}` where `binding` is a 1-bit flag for the persistent/bound distinction. We trade a small impurity of the encoding function for a 1-character routing decision that would otherwise require an external lookup.

The partitioning scheme's fourth clientId range — committed ephemeral $[2^{52}, 3 \cdot 2^{51})$ — has no entity

prefix. That range holds the clientIds of operations produced at liminal commit[6]: in shadow-document implementations of liminality, where commit is a rewrite of the session's tentative clientId block into the committed range; in single-document implementations, where commit writes new operations at fresh committed-range clientIds while leaving the tentative originals as marked-undone ops in the liminal range. In both variants the committed range serves as a priority tier that preempts liminal and regular writes in conflict resolution. Committed-range operations host scalar writes to existing entity shells rather than creating new entities — entities themselves retain the `l-` or `p`-prefixed UUID under which they were first materialized — so no `c`-prefix is needed in the identifier encoding.

Body encoding. The body packs 83 bits into 14 base-63 characters (capacity: $\lfloor 14 \cdot \log_2 63 \rfloor = \lfloor 83.68 \rfloor = 83$ bits):

- `a`: upper 19 bits of clientId payload (clientId minus range base)
- `b`: lower 32 bits of clientId payload
- `c`: 32-bit Lamport clock

Combined value: $a \cdot 2^{64} + b \cdot 2^{32} + c$. Total: $19+32+32 = 83$ bits.

These three unsigned integers form a big number represented as three uint32 chunks. Encoding extracts base-63 digits via long division across chunks; decoding reconstructs via long multiplication:

```
encode: for i = 13 downto 0:
          carry = 0
          for j = 0 to 2:
            cur = carry * 2^32 + chunk[j]
            chunk[j] = floor(cur / 63)
            carry = cur mod 63
          digit[i] = carry

decode: for i = 0 to 13:
          carry = digit[i]
          for j = 2 downto 0:
            cur = chunk[j] * 63 + carry
            chunk[j] = cur mod 2^32
            carry = floor(cur / 2^32)
```

All intermediates stay within JavaScript's safe integer range: max intermediate is $62 \cdot 2^{32} + (2^{32} - 1) \approx 2.7 \cdot 10^{11}$, well below 2^{53}.

Alphabet: `a-zA-Z0-9_` — 63 characters. Valid in JavaScript identifiers (`obj.pRxK4mN7pQ2wJbL`), CSS class names, HTML attributes, URL components, and JSON keys — all without escaping. The `-` character was excluded specifically to preserve JavaScript identifier validity.

5.3 Visual Dispersion

Sequential entity creation produces sequential `{clientId, clock}` pairs. Without treatment, consecutive identifiers differ only in trailing characters — making visual identification difficult and git diffs noisy.

We apply a 4-round balanced Feistel network on the lower 64 bits `(b, c)` before encoding. The upper 19 bits pass through unscrambled (they derive from the random base and rarely change between entities).

```
Round function:
  f(half, key) = murmurhash2_finalizer(half XOR key)
  where murmurhash2_finalizer(h) =
    h *= 0x5bd1e995; h ^= h >>> 13;
    h *= 0x5bd1e995; h ^= h >>> 15

Round keys:
  [0x6a09e667, 0xbb67ae85,
   0x3c6ef372, 0xa54ff53a]
  (fractional parts of sqrt(2,3,5,7)
   -- nothing-up-my-sleeve constants)
```

Properties: bijective, invertible in $O(1)$, empirically dispersive for sequential inputs. Bijectivity of the full 83-bit map follows from the Cartesian product of an identity on the upper 19 bits (payload high) and a Feistel permutation on the lower 64 bits `(b, c)`; the Feistel structure is itself bijective as a standard result[7]. Four rounds is one beyond the Luby–Rackoff PRP threshold (three) — sufficient for the dispersion target without premature cryptographic weight. The construction is not cryptographic (round keys are public constants); any invertible permutation with adequate avalanche would work.

Deterministic-range identifiers (`d` prefix) skip the Feistel step: they are already content-addressed hashes with inherent dispersion.

5.4 Resolution

1. Read prefix → determine lifecycle stage and clientId range base.
2. Decode body → `{a, fL, fR}`.
3. Feistel decrypt → `{b, c} = {payloadLo, clock}` (skip for `d`).
4. Reconstruct: clientId $= a \cdot 2^{32} + b +$ base.
5. Binary search StructStore for `(clientId, clock)`.

Step 5 uses the CRDT runtime's native operation lookup — in Yjs, a binary search over the per-clientId sorted item array. No auxiliary index or mapping table.

Complexity: $O(1)$ decode (14 multiply-accumulate steps + 4 Feistel rounds) + $O(\log m)$ binary search where m is the operation count for the relevant clientId.

5.5 Encoding Properties

P1. Self-resolving. The identifier contains the complete physical address. Any peer with the relevant operations can resolve it — no registry required. Snapshots with cross-references are self-contained.

P2. Uniqueness. The encoding is a bijection from `{clientId, clock, binding}` to 15-character strings — distinct inputs produce distinct identifiers, where `binding` is the 1-bit `b`/`p` flag discussed in §5.2. For user-created entities, uniqueness inherits from the CRDT's own replica isolation guarantee — subject to the creating operation being permanent, an invariant preserved through local undo/redo by the discipline described in Appendix A. For deterministic entities (`d` prefix), identity is content-addressed — two peers computing the same genesis path produce identical identifiers (intentional convergence, not collision resistance). The genesis hash uses 51 bits; birthday collision probability for 10K genesis paths is $\sim 2.2 \times 10^{-8}$.

P3. Prefix discrimination. Lifecycle stage is visible from the first character. Filtering, routing, and access control operate on the prefix without decoding. Examples: excluding genesis entities from the undo stack (one character compare), filtering ephemeral previews from persistence, blocking reparenting of cloned references.

P4. Stable across sync and undo. The identifier encodes the creating operation's coordinates — identical on every peer that has received that operation. No consensus or central authority. Stability through local undo/redo is not automatic in operation-based CRDTs (naive undo would remove the creating operation and redo would mint a new one); it requires the discipline described in Appendix A.

P5. Compact. 15 ASCII characters encode 83 bits of structured information plus lifecycle metadata. UUID v4 is 36 characters encoding 122 random bits with no structural content. For wider identifier systems (Automerge's 128-bit actorIds + 64-bit counters = 192 bits), the encoding scales to $\lceil 192/5.98 \rceil + 1 = 33$ characters — at which point the compactness advantage over UUID v4 diminishes.

5.6 Lifecycle-Aware Routing

The prefix character enables constant-time routing without decoding. From the production system:

- **Genesis exclusion from undo stack:** Genesis operations have `d`-prefixed identifiers. A `stack-item-added` listener strips the genesis client from the captured stack item via one character comparison: genesis entities are never a target of Ctrl-Z. This is distinct from entity-shell preservation (Appendix A) — one filters entire clientIds by prefix at stack-capture time, the other protects individual entity-shell Items at delete time by structural position.
- **Liminal isolation:** `l`-prefixed entities are ephemeral previews. The sync layer filters them from persistence: `if (id[0] === 'l') skipPersistence()`.
- **Clone protection:** `b`-prefixed entities cannot be reparented. The binding layer enforces this by prefix, reading the runtime flag directly from the identifier rather than consulting an external map.

Genesis exclusion and liminal isolation depend on the companion namespace partitioning scheme[5]. Clone protection additionally uses the prefix as a lifecycle channel orthogonal to range, as noted in §5.2.

5.7 Deterministic Genesis Convergence

Genesis entities (schema containers, type sub-maps) use the `d` prefix with content-addressed clientIds derived from a pure function of (type, path), as described by the companion deterministic genesis paper[2]. Each genesis element is produced in a throwaway CRDT document — guaranteeing clock = 0. Two independent peers computing the same structural skeleton produce byte-identical operations and therefore identical identifiers.

This is a hybrid of coordinate-addressed identity (for user-created entities) and content-addressed identity (for structural entities), unified under a single encoding with prefix discrimination. The combination echoes content-addressed storage (IPFS CIDs use self-describing prefixes for a similar purpose) while preserving the mutability that CRDT entities require.

6 Applications

6.1 Completeness Checking

A peer with partial replication has some subset of the document's operations. Running the §3.2 traversal against schema and root produces the `missing` set — references that cannot be resolved locally. This is the operational completeness signal. The check is cheap

($O((E+R)\cdot \log m)$) and runs on every sync boundary in production.

6.2 Dependency Loading Without Manifests

A document references entities from library documents (`[uuid, depId]` cross-document references). On load:

1. Run traversal from root via §3.2.
2. Collect missing `(refId, depId)` tuples.
3. Load dependencies for each missing `depId`.
4. Re-traverse (dependencies may reference transitive dependencies).
5. Repeat until the reference graph is closed.

The reference graph produces the **loading specification** — which dependencies are needed. A separate resolution service (mapping `depId` to a fetchable blob) is still required. The contribution is making the specification derivable from the document rather than manually maintained in a manifest.

Cycle termination. Circular dependencies terminate via the `visited` set; the loading loop additionally tracks `loadedDocs` to avoid re-fetching.

6.3 Partial Sync Scope Derivation

A large document is split into sections (Yjs subdocuments or equivalent). Loading section A:

1. Sync section A.
2. Traverse from A's entities.
3. Discover references into sections B and C.
4. Load B and C (or queue for lazy loading with a placeholder).

The sync scope derives from the reference graph, not a manual specification. A cross-section reference added by a peer automatically expands the sync scope on all other peers at their next check.

6.4 Portable Snapshots

Because identifiers are self-resolving, serialized CRDT state is self-contained: cross-references resolve against any peer's operation log without a companion registry artifact. Export, import, and offline reconciliation inherit this property — the reference graph survives serialization without any companion index.

7 Constraints

Constraints of the architecture and of the specific encoding are enumerated together. Some apply to both.

C1. Schema requirement. The traversal needs to know which fields contain references. Systems with complete schemas (Plexus decorators, TypeScript-typed Automerge) satisfy this. Schema-free CRDT usage (raw Y.Maps with arbitrary keys) cannot enumerate references without application guidance.

C2. Operation-based CRDTs only. The encoding requires a CRDT where each operation has a unique `{replicaId, sequence}` pair and the operation log supports efficient lookup by those coordinates. The architecture generalizes to any CRDT with decodable entity identifiers, but the encoding of §5 is operation-based. State-based CRDTs and position-based sequence CRDTs (Logoot/LSEQ) are out of scope.

C3. Decodable identity for cross-document. Intra-document traversal works with any resolvable reference format. Cross-document traversal requires references that encode their document of origin — the `[uuid, depId]` pattern or equivalent.

C4. Immutable identity. The identifier is derived from the creating operation and cannot change. Lifecycle transitions (e.g., liminal → committed) produce a new operation with a new identifier; the relationship between identifiers is recoverable from the clientId namespace arithmetic.

C5. Cooperative deployment. The encoding is not authenticated. A malicious peer can craft identifiers that decode to arbitrary `{clientId, clock}` pairs. Prefix-based routing is cosmetic, not a security boundary — the prefix is not cryptographically bound to the payload. Enforcement requires a validation layer (server-side relay, authenticated sync) outside the encoding. This is comparable to how CRDTs generally assume authenticated peer identity.

C6. Information exposure. Given a set of identifiers from a single peer, an attacker recovers the complete Lamport clock sequence — revealing the total order of entity creation, including gaps that indicate non-entity operations. Acceptable for collaborative editing where operation metadata is visible to all peers; potentially unacceptable for privacy-sensitive deployments.

C7. Operation permanence dependency. The encoding assumes the creating operation for an entity remains addressable in the operation log. Two mechanisms can violate this: (a) runtime garbage collection of operations, and (b) undo-driven deletion of the creating operation itself. The implementation addresses

both with one discipline — the entity-shell protection described in Appendix A, plus configuring Yjs with `gc: false` to retain the operation log. Systems that cannot disable GC must either prevent collection of entity-shell operations or treat unresolvable identifiers as dangling references.

C8. Schema evolution. A traversal running an old schema misses references added in a newer schema version. Liveness and completeness are sound only for the schema version they were run with. Cold-storage offload (§4) is the safer disposition under schema drift: an entity offloaded under an old schema that added a reference field under a new schema will simply be promoted back on the first traversal with the new schema.

C9. Inverse references. Detecting newly-dangling references on entity mutation (e.g., a parent field being cleared) requires either full re-traversal or an inverted reference index. Additions are incremental ($O(refs\_per\_entity \cdot \log m)$); deletions are either batched or indexed.

C10. Cold-tier resolution service. The offload tier needs a service (local disk, blob store, CDN) that maps UUIDs to serialized shells. Failure of this service downgrades offload to effective loss — systems should treat cold-tier availability as a durability requirement, not a performance optimization.

C11. Clock and identifier width. The 32-bit clock supports ∼4.3 billion operations per client — sufficient for collaborative editing but not unbounded. The 51-bit clientId + 32-bit clock = 83-bit payload is specific to JavaScript's safe integer constraint ($2^{53} - 1$). Platforms with wider integers can use more bits. The encoding generalizes: base-63 at ∼6 bits/character, parameterized by total payload width.

C12. Eventual, not immediate. In a live system, new references are created continuously. Traversal provides a point-in-time snapshot; between runs, transient incompleteness is normal and resolves on the next sync + traversal cycle.

8 Related Work

8.1 Architectural Lineage

Shapiro et al.[8] formalize referential integrity under causal consistency — preventing dangling references via compensating operations. Integrity is a write-time constraint (prevent violations); the present paper's completeness and liveness are read-time properties of partial replicas. The two are orthogonal.

Kleppmann's replicated tree[9] uses a TRASH node for deletion semantics — tree reachability determines liveness in a single-parent hierarchy. Our reference graph generalizes this to multi-parent DAGs with schema-declared edges; the cold-storage offload disposition is a further departure.

Guerreiro et al.[10] formalize CRDT partial replication by decomposing objects into "particles" and "shard sets" with per-object causal metadata. Their sharding is application-specified. Our sharding is reference-derived: the graph walk produces the shard specification.

Dolan[1] proves that algebraic undo of general CRDTs is impossible beyond counters. We treat this as a design constraint rather than a limit: the cold-storage offload disposition specifically avoids invoking a "delete" operation that would need to be undone.

Content-addressed storage (IPFS, Merkle-DAG) offers a useful comparison. Offload indexed by decodable UUID is structurally similar — portable, self-verifying objects addressed by coordinate — but our UUIDs are coordinate-derived (CRDT operation position) rather than content-derived (hash of payload). Coordinate-addressing preserves entity identity under mutation; content-addressing preserves content integrity under immutability. Both are useful; we pay the former's price.

Yjs subdocuments[11] provide sync scoping at document granularity without cross-subdocument reference tracking; developers manually specify which subdocuments to load. **Automerge**[12] treats each document as a self-contained sync unit, and its cross-document references are unmanaged — although Automerge objectIds could support intra-document completeness checking without the specific encoding of §5.

The general principle — follow references, fetch what's missing — is well-established outside CRDT research. Module bundlers (webpack, esbuild) and package managers (apt, npm) derive specifications from import graphs. The CRDT-specific contribution is that references are embedded in mutable replicated state and change during sync, so the specification is never finalized at build time.

8.2 Identity Encoding Lineage

Automerge objectIds[12] are the closest prior art for coordinate-derived identity — each Automerge object receives an `objectId` derived from its creating operation's `{actorId, counter}`, formatted as "`{counter}@{actorId}`". The same core insight. Our encoding adds fixed-length (15 characters regardless of identifier width), lifecycle prefixes, URL-safe alphabet,

and visual dispersion — engineering refinements rather than new mechanism.

Yjs[13] provides the StructStore (sorted per-client operation log) that makes $O(\log n)$ resolution possible, and `RelativePosition` applies coordinate-derived identity to positions inside sequences. **Automerge's** `Cursor` and **Loro's** `ContainerID` are direct analogues of `RelativePosition` — coordinate-derived identifiers for positions or containers, surfaced at the library API level. None surfaces an encoding intended for use as application-level entity identifiers in a reference graph; our contribution extends the underlying mechanism from positions/containers to entities.

Merkle-CRDTs[14] use content-addressed hashing for operation identity (deduplication), not entity identity. Merkle-CRDT hashes are content-derived (hash of operation payload); our identifiers are coordinate-derived (position in the operation log). Content-derived identifiers enable deduplication; coordinate-derived identifiers enable $O(\log n)$ lookup in a sorted store. Our `d`-prefix genesis identifiers are content-addressed, making the system a hybrid of both approaches.

UUIDs (RFC 9562) provide universal uniqueness through randomness (v4), timestamps (v7), or names (v5). None encode CRDT coordinates. A UUID v4 used as entity identity in a CRDT system requires the auxiliary mapping our encoding avoids.

8.3 Companion Techniques

Semantic clientId partitioning[5] provides the namespace ranges that make lifecycle prefixes meaningful. The encoding of §5 depends on[5] for its lifecycle features; the core encoding (collapsing `{replicaId, clock}` into a fixed-length string) is independent.

Deterministic genesis[2] provides the structural skeleton at identical paths on every peer — these are the type sub-maps that anchor §2.1's flat-shell architecture. The `d` prefix of §5.2 is the encoded form of genesis clientIds.

Liminal state[6] uses the `l` prefix and the *committed ephemeral* range (§5.2) for gesture-scoped operations held locally and their commit-time rewrite. The ephemeral-to-committed identifier rewrite discussed in §5.2 is described there.

Composite entity-addressed keys[3] extend the reference primitive into map-key positions, using the encoding of §5 for the entity-reference components of composite keys.

Local-first software[15] frames the paradigm in which CRDT-native applications operate. The architectural choices here — flat shells, pointer references, reachability-based liveness — are consequences of taking local-first seriously: working-set cost scales with the reachable graph, and cold-storage archives support long-term durability without coordination.

9 Conclusion

Modeling a CRDT document as a flat registry of entity shells connected by a decodable reference graph is a structural choice about where identity and edges live, and the systems-level consequences follow directly. The same graph walk that enumerates missing references for sync completeness also produces the partial-replication load set and the liveness partition for storage tiering — these are three uses of one traversal, not three separately engineered subsystems.

Reachability replaces tombstones as the liveness signal, and cold-storage offload replaces deletion as the disposition. The result is a GC alternative that respects Dolan's impossibility result[1] by construction: nothing is deleted, so nothing needs to be undone. Lifetime storage still grows, but in tiers: the working-set cost scales with the reachable graph while the archive cost scales with lifetime entity count, and the two are decoupled by a service boundary that the application already needs for partial replication.

Flat entities, pointer references, and a first-class reference graph are the architectural load-bearing choices. The concrete encoding of §5 is engineering refinement over prior coordinate-derived schemes (Automerge objectIds, Yjs RelativePosition) — fixed length, lifecycle prefixes, URL-safety, visual dispersion — and the undo discipline of Appendix A is a runtime invariant specific to Yjs-style op-based CRDTs with general UndoManagers. The three layers are complementary: the architecture is where the theoretical claims sit, the encoding and runtime discipline are portable engineering choices that other op-based runtimes could realize differently.

A Undo Handling for Coordinate-Derived Identity in Yjs

This appendix documents a set of Yjs-specific workarounds required to make the encoding of §5 behave correctly under local undo/redo. They are implementation notes, not research contributions — similar concerns likely apply to other op-based CRDT runtimes with general UndoManagers (Loro, Automerge

with undo extensions). The underlying insight is one paragraph; the rest is Yjs plumbing.

A.1 The Problem

Properties P2 (uniqueness) and P4 (stable across sync) of §5.5 assume the creating operation for an entity remains addressable on every peer that has received it. Sync honors this assumption — operations are not retracted. Local undo does not. In Yjs (and op-based CRDTs generally), undo is implemented by the UndoManager generating a DELETE operation against the target Item, and redo by re-inserting with a fresh `{clientId, clock}`. Naively applied to an entity, this would:

1. Undo the creating operation → the entity becomes unresolvable (dangling identifier in any referring operation).
2. Redo → a new creating operation with a new clock → a different identifier for what the user perceives as the same entity.

Either outcome violates the encoding's contract. Because the identifier is derived from the creating operation's coordinates, identity stability requires that the creating operation itself be structurally unremovable from the originator's perspective.

A.2 The Discipline: Append-Only Entity Shells

We resolve this with a runtime discipline: entity identity is append-only even when the rest of the CRDT is not. Three mechanisms cooperate.

M1. Structural shell protection. Entities are stored as entries in a type-indexed container — one container per entity class, with `uuid` → `shell` bindings, located at well-known paths established by deterministic genesis [2]. The UndoManager is configured with a `deleteFilter` that rejects any deletion whose parent is one of these containers. The check is structural, not an allowlist: by the genesis structure's construction any type-indexed container is an entity-shell container, and the protection follows from the structural shape rather than from application-maintained metadata.

M2. Materialization-clock watermark. A shell survives undo, but field writes *inside* it must remain undoable — otherwise the entity would be frozen at creation. Each entity records its creating client and clock at the end of its initialization phase (when the shell and its seed attributes are integrated into the document). Items inside the shell whose `(client, clock)` match the creating client and whose clock is below the watermark are "creation content" and protected by the same `deleteFilter`; items at or above the watermark are post-creation modifications and remain undoable. This distinguishes the invariant floor (exists-with-initial-values) from the mutable ceiling (current field values) structurally, without tagging individual operations.

M3. Deferred creation boundary. Without intervention, operations that materialize an entity and operations that subsequently edit it would land in the same UndoManager stack item — one Ctrl-Z would destroy both. On transaction exit, if any entity was created, we schedule a stack-boundary call for the microtask-after-next, cancelable by a new transaction opening first. Creation thus becomes its own undo step; subsequent edits merge under the normal capture timeout. The user experience: early undos revert edits; the final undo detaches the entity from its parent but does not destroy its identity. Redo replays the edit chain on the same shell.

Together M1–M3 give an **identity floor**: once created, an entity's address is a permanent coordinate in the operation log. Parent wiring (which container references the entity) reverts normally — the shell simply becomes unreferenced, persisting as addressable but detached state. Garbage is recovered through the reachability-based collection of §4 at snapshot time, not through undo-driven deletion.

A.3 Scope Note

The discipline operates on the originator's local undo stack. Remote peers receive only the mutations that survive deletion filtering; the UndoManager typically uses an origin-tracked scope so that peer-originated operations are not targetable by local undo in the first place. The discipline is local to the runtime. No change to the CRDT specification, the wire format, or peer behavior is required — remote peers observe a document in which the creating operation is never retracted, which is the pre-undo invariant the encoding already assumes.

An illustrative snippet in Yjs terms:

```
// M1, abbreviated: reject deletions whose
// parent is an entity-shell container
deleteFilter: (item) =>
  isEntityShellContainer(item.parent)
    ? false
    : /* M2 check */ true;
```

A.4 Portability Beyond Yjs

The three mechanisms map to any op-based CRDT runtime with a general UndoManager:

- M1 requires a hook that decides, per-operation, whether undo is allowed. Yjs exposes `deleteFilter`; Loro and Automerge-with-undo would need equivalent extension points.
- M2 requires the runtime to expose enough operation metadata (creating client, clock) for the watermark check. Most op-based CRDTs do.
- M3 requires the runtime to expose a capture-boundary API. Yjs has `stopCapturing()`; other runtimes vary in how they group operations into undo stack items.

Where these extension points are absent, the discipline still applies in principle but must be implemented via whatever patching is available — e.g., monkey-patching the undo generation path, or wrapping the runtime with a custom undo facade. The mechanism isn't novel; the need for it is inherent to combining coordinate-derived identity with general-purpose undo.

References

[1] Stephen Dolan. The only undoable CRDTs are counters. *arXiv:2006.10494*, 2020.

[2] Plexus Project. Deterministic genesis: Coordination-free structural initialization for operation-based CRDTs. Preprint, 2026.

[3] Plexus Project. Composite entity-addressed keys for CRDT maps. Preprint, 2026.

[4] Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. A comprehensive study of convergent and commutative replicated data types. *INRIA RR-7506*, 2011.

[5] Plexus Project. Adapting categorical CRDT lifecycle to op-based CRDTs via replica-identifier partitioning. Preprint, 2026.

[6] Plexus Project. Liminal state: Deferred-persistence lifecycles for operation-based CRDTs. Preprint, 2026.

[7] Michael Luby and Charles Rackoff. How to construct pseudorandom permutations from pseudorandom functions. *SIAM Journal on Computing*, 1988.

[8] Marc Shapiro, Annette Bieniusa, Peter Zeller, and Gustavo Petri. Ensuring referential integrity under causal consistency. 2018.

[9] Martin Kleppmann, Dominic P. Mulligan, Victor B.F̃. Gomes, and Alastair R. Beresford. A highly-available move operation for replicated trees. *IEEE Transactions on Parallel and Distributed Systems*, 2021.

[10] João Guerreiro, Paulo Sérgio Almeida, and Carlos Baquero. Conflict-free partially replicated data types. In *Proc. IEEE SRDS*, 2015.

[11] Yjs Authors. Yjs subdocuments. https://docs.yjs.dev/api/subdocuments, 2024.

[12] Martin Kleppmann and Alastair R. Beresford. A conflict-free replicated JSON datatype. *IEEE TPDS*, 2017.

[13] Yjs Authors. Yjs INTERNALS. https://github.com/yjs/yjs/blob/main/INTERNALS.md, 2024.

[14] Héctor Sanjuán, Samuli Pöyhtäri, and Pedro Teixeira. Merkle-CRDTs: Merkle-DAGs meet CRDTs. Protocol Labs, 2020.

[15] Martin Kleppmann, Adam Wiggins, Peter van Hardenberg, and Mark McGranaghan. Local-first software: You own your data, in spite of the cloud. Onward!, 2019.